

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки і комп'ютерних технологій

Звіт
про виконання лабораторної роботи №3
з курсу «Хмарні технології та великі дані»
на тему:
«Retry (Повтор)»

Виконав:
студент групи ФсІ-42
Трофимов Володимир
Перевірив:
ас. Керод Т. І.

Зміст

Тема.....	3
Мета	3
Вхідні дані	3
Завдання.....	3
Критерії виконання роботи.....	4
Теоретичні відомості	5
Структура програмної реалізації	6
Тестування програмної реалізації.....	6
Висновок	10
Код програми	11

Тема

Підвищення надійності взаємодії з нестабільними сервісами за допомогою патерну Retry.

Мета

Дослідження принципу роботи патерну Retry (Повтор). Необхідно створити механізм безпечного повторного виконання асинхронних операцій у разі виникнення транзитних збоїв, реалізувати різні стратегії пауз (constant, exponential, exponentialJitter), фільтрацію помилок (retryOn), а також продемонструвати працездатність системи шляхом перевірки через HTTP-запити.

Вхідні дані

Параметр	Значення	Призначення
maxAttempts	4	Максимальна кількість спроб виконання операції (включаючи першу)
strategy	exponential	Стратегія обчислення затримки між спробами (constant, exponential, exponentialJitter)
baseDelayMs	500 мс	Базовий час очікування перед повторною спробою
multiplier	2	Множник експоненційного зростання затримки
maxDelayMs	30 000 мс	Максимально допустимий ліміт затримки (стеля очікування)
maxJitterMs	250 мс	Максимальна випадкова часова добавка (джиттер) для усунення синхронізації повторів
retryOn	Коди 5xx або предикат	

Завдання

Розробити програмну реалізацію патерну Retry мовою TypeScript для підвищення стійкості застосунку під час виникнення тимчасових (транзитних) помилок.

Реалізація повинна забезпечувати:

- створення узагальненого класу або утиліти Retriyer з типізованим методом execute(fn);
- підтримку конфігурації кількості спроб maxAttempts;

- реалізацію трьох стратегій розрахунку паузи: фіксованої (constant), експоненційної (exponential) та експоненційної з випадковим шумом (exponentialJitter);
- фільтрацію помилок через параметр retryOn (перелік кодів, типи винятків або предикат);
- негайне переривання виконання та прокидання помилки, якщо вона визначена як non-retryable;
- завершення роботи з генерацією винятку після вичерпання ліміту maxAttempts;
- логування спроб у реальному часі через колбек onRetry із фіксацією номера спроби, часу очікування та коду помилки;
- модульне тестування з використанням фейкових таймерів (fake timers) для миттєвої перевірки часових затримок без реального очікування;
- створення локального демо-сервера на базі Node.js для емуляції ненадійного сервісу та ручного тестування через curl;
- реалізацію тестових ендпоінтів: тимчасово недоступного (/mock/flaky) та фатального (/mock/fatal).

Критерії виконання роботи

Лабораторна робота вважається виконаною за умови проходження автоматизованих тестів та успішної ручної перевірки через HTTP-запити за такими критеріями:

1. Реалізація механізму Retry

Програмний модуль приймає будь-яку асинхронну функцію `fn: () => Promise<T>` та виконує її повторно лише в разі виникнення помилки, визначеної правилом `retryOn`.

2. Успіх з першої спроби

Якщо операція виконується успішно з першого разу, функція не створює пауз, лічильник спроб дорівнює 1, а результат повертається негайно.

3. Успіх з n -тої спроби

Якщо сервіс падає $n - 1$ разів через тимчасові збої (наприклад, HTTP 503), механізм робить відповідну кількість пауз і повертає успішний результат після n -го виклику.

4. Вичерпання спроб (`maxAttempts`)

Після досягнення ліміту спроб чергова спроба не ініціюється, а система викидає останню отриману помилку. Сумарний час затримок у тестах має точно відповідати розрахунковому значенню.

5. Поведінка при `non-retryable` помилках

При отриманні незворотної помилки (наприклад, HTTP 400 Bad Request або помилки бізнес-логіки) повторні спроби не виконуються, затримка не нараховується, а помилка прокидається миттєво після першого виклику.

6. Коректність стратегій затримок

- Для `constant`: затримка є сталою і дорівнює `baseDelayMs`.

- Для exponential: затримка зростає за формулою $\text{baseDelayMs} \times \text{multiplier}^{(\text{attempt}-1)}$ з обмеженням maxDelayMs .
 - Для exponentialJitter: до експоненційного значення додається псевдовипадкова величина в діапазоні $[0, \text{maxJitterMs}]$.
7. Ефективність автоматизованих тестів
- Використання fake timers дозволяє виконати весь набір unit-тестів (включаючи перевірку сумарних затримок у десятки секунд) менш ніж за кілька сотень мілісекунд реального часу.
8. Ручна демонстрація роботи
- Через термінал (curl) або браузер продемонстровано роботу всіх ключових сценаріїв:
- успішне виконання з 1-ї спроби ($\text{failCount}=0$);
 - відновлення після серії збоїв ($\text{failCount}=2$, $\text{maxAttempts}=4$);
 - вичерпання ліміту спроб ($\text{failCount}=5$, $\text{maxAttempts}=3$);
 - миттєва відмова при запиті до фатального маршруту (`/mock/fatal`);
 - різниця між детермінованою експонентою та експонентою з джитером.

Теоретичні відомості

Retry (Повтор) — це патерн проєктування розподілених та хмарних систем, призначений для прозорої обробки транзитних (короткочасних) збоїв мережі та зовнішніх сервісів.

Транзитні помилки (наприклад, короткочасне переривання з'єднання, перевантаження віддаленого сервера HTTP 503, відповідь 429 Too Many Requests або тайм-аут шлюзу HTTP 504) зазвичай усуваються самостійно протягом кількох десятих часток секунди. Патерн Retry автоматично повторює неуспішний запит через певні проміжки часу, ізолюючи збій від кінцевого користувача або клієнтського коду.

Ключовим аспектом правильної реалізації патерну є вибір стратегії очікування:

- Constant backoff: прості фіксовані інтервали. Може створювати надлишковий тиск на віддалену систему.
- Exponential backoff: геометричне збільшення часу очікування після кожної невдачі, що дає перевантаженій системі час на стабілізацію.
- Full / Equal Jitter: додавання випадкового часового зміщення до експоненційної паузи. Необхідне для запобігання проблемі «Thundering Herd» (коли сотні синхронізованих клієнтів одночасно надсилають повторні запити в одну й ту саму мілісекунду).

Не менш важливою є класифікація помилок: патерн повинен розрізняти транзитні помилки (серверні 5xx) від детермінованих помилок клієнта

(клієнтські 4xx), оскільки повторення невалідного запиту безглузде і лише витрачає ресурси інфраструктури.

Структура програмної реалізації

Програмне рішення розроблене мовою TypeScript для платформи Node.js і має модульну структуру:

Модуль патерну (src/retry.ts): містить визначення типів конфігурації, функцію розрахунку пауз calculateDelay з урахуванням обраної стратегії та джитера, допоміжні функції перевірки retryOn, а також універсальний клас Retryer із методом execute().

Сервер для перевірки (src/server.ts): легкий HTTP-сервер на базі node:http, який реалізує тестові маршрути:

- /mock/flaky — імітатор нестабільного сервісу з налаштовуваною кількістю збоїв;
- /mock/fatal — імітатор постійної клієнтської помилки (HTTP 400);
- /demo/run — точка входу для тестувальника, яка запускає внутрішній екземпляр Retryer, детально логує кроки в консоль та повертає підсумковий JSON-звіт.

Тестування програмної реалізації

Тестування програмної реалізації патерну Retry виконується комбінованим шляхом: автоматизовано за допомогою unit-тестів та вручну шляхом відправлення HTTP-запитів через утиліту curl (або веббраузер) до демонстраційного сервера.

Перевіримо роботу патерну в реальному часі. Для цього запускається локальний Node.js HTTP-сервер. Сервер починає прослуховувати порт 3000, надаючи тестові ендпоінти для симуляції збоїв (/mock/flaky, /mock/fatal) та демонстраційний маршрут /demo/run.

Для першого експерименту перевіряється базовий сценарій: успішне виконання операції з першої спроби без виникнення збоїв (failCount = 0). Через термінал надсилається запит:

```
curl "http://localhost:3000/demo/run?failCount=0"
```

У цьому випадку цільовий ендпоінт відразу повертає статус 200 ОК. У консолі сервера фіксується лише один виклик, масив затримок залишається порожнім (delays: []), а сумарна кількість спроб дорівнює totalAttempts: 1.

```
admin@DESKTOP-BQIH00U MINGW64 /d/4_1/Cloud_Technologies/Lab_3 (main)
$ npm run dev
OK.

admin@DESKTOP-BQIH00U MINGW64 /d/4_1/Cloud_Technologies/Lab_3 (main)
$ npm run dev
> retry-pattern-lab@1.0.0 dev
> tsx src/server.ts

Retry demo server listening at
http://localhost:3000
[Attempt 1] Succeeded with 200
OK.

admin@DESKTOP-BQIH00U MINGW64 /d/4_1/Cloud_Technologies/Lab_3 (main)
$ curl "http://localhost:3000/demo/run?failCount=0"
{"success":true,"totalAttempts":1,"delays":[],"successStatus":200,"finalResponse":{"status":200,"body":{"ok":true,"attempt":1,"message":"Flaky service recovered"}}}
```

Рис.1 – Робота Retry при успішному виконанні з першої спроби ($failCount = 0$)

Наступним експериментом перевіряється відновлення після транзитних збоїв (успіх з n -ї спроби) з використанням експоненційної стратегії затримок ($strategy = exponential$). Встановлюються параметри:

- $maxAttempts = 4$
- $failCount = 2$
- $baseDelayMs = 500$
- $multiplier = 2$

Запит виконується командою:

curl

"http://localhost:3000/demo/run?strategy=exponential&maxAttempts=4&failCount=2&baseDelayMs=500&multiplier=2"

Під час перших двох викликів сервіс повертає помилку 503 Service Unavailable. Механізм Retriyer перехоплює їх і виконує паузи: після першої спроби — 500 мс (500×2^0), після другої — 1000 мс (500×2^1). На третій спробі сервіс відновлюється, повертаючи 200 OK. Підсумковий JSON містить $totalAttempts: 3$ та масив затримок $delays: [500, 1000]$.

```
admin@DESKTOP-BQIH00U MINGW64 /d/4_1/Cloud_Technologies/Lab_3 (main)
$ npm run dev
> retry-pattern-lab@1.0.0 dev
> tsx src/server.ts

Retry demo server listening at
http://localhost:3000
[Attempt 1] Succeeded with 200
OK.
[Attempt 1] Failed with 503. Retrying in 500ms...
[Attempt 2] Failed with 503. Retrying in 1000ms...
[Attempt 3] Succeeded with 200
OK.

admin@DESKTOP-BQIH00U MINGW64 /d/4_1/Cloud_Technologies/Lab_3 (main)
$ curl "http://localhost:3000/demo/run?strategy=exponential&maxAttempts=4&failCount=2&baseDelayMs=500&multiplier=2"
{"success":true,"totalAttempts":3,"delays":[500,1000],"successStatus":200,"finalResponse":{"status":200,"body":{"ok":true,"attempt":3,"message":"Flaky service recovered"}}}
```

Рис.2 – Відновлення після транзитних помилок з експоненційним backoff ($failCount = 2$)

Далі перевіряється сценарій вичерпання максимальної кількості спроб ($maxAttempts$). Для наочності обирається стала стратегія затримок ($strategy = constant$):

- $maxAttempts = 3$

- failCount = 5
- baseDelayMs = 400

Запит виконується командою:

curl

"http://localhost:3000/demo/run?strategy=constant&maxAttempts=3&failCount=5&baseDelayMs=400"

Цільовий сервіс генерує помилку 5 разів поспіль, проте Retryer зупиняє виконання рівно після 3 спроб. Здійснюються дві фіксовані паузи по 400 мс, після чого клієнту повертається фінальний статус 503 та повідомлення про вичерпання спроб із totalAttempts: 3 та delays: [400, 400].

Рис.3 – Поведінка Retry при вичерпанні ліміту спроб (*maxAttempts* = 3, *failCount* = 5)

Також перевіряється поведінка системи при виникненні незворотних (non-retryable) помилок. Для цього запит спрямовується на ендпоінт фатального збою (target = fatal):

curl "http://localhost:3000/demo/run?target=fatal&maxAttempts=5"

Ендпоінт повертає клієнтську помилку 400 Bad Request. Оскільки налаштований предикат `retryOn` відсікає помилки клієнта і дозволяє повторювати лише серверні збої 5xx, механізм миттєво перериває виконання без пауз та повторів (totalAttempts: 1, delays: []).

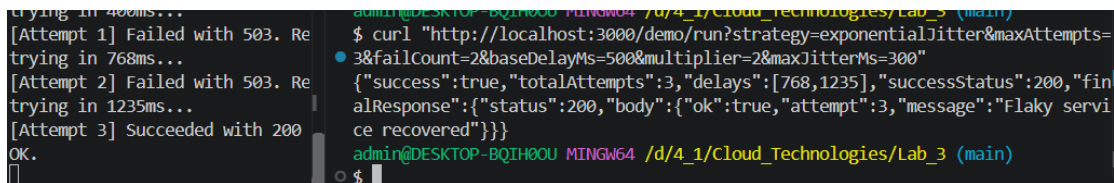
Рис.4 – Миттєва зупинка на non-retryable помилці (HTTP 400 Bad Request)

Окремо перевіряється робота стратегії експоненційного очікування з додаванням випадкового шуму (strategy = exponentialJitter) для запобігання синхронізації повторних запитів:

curl

"http://localhost:3000/demo/run?strategy=exponentialJitter&maxAttempts=3&failCount=2&baseDelayMs=500&multiplier=2&maxJitterMs=300"

У цьому режимі до розрахункових експоненційних значень (500 мс та 1000 мс) додається псевдовипадкове число в діапазоні $[0, 300]$ мс, що наочно відображається у консольному журналі сервера та фінальній JSON-відповіді з унікальними затримками.



```
trying in 400ms...  
[Attempt 1] Failed with 503. Re  
trying in 768ms...  
[Attempt 2] Failed with 503. Re  
trying in 1235ms...  
[Attempt 3] Succeeded with 200  
OK.  
$  
admin@DESKTOP-BQIH00U MINGW64 /d/4_1/Cloud_Technologies/Lab_3 (main)  
$ curl "http://localhost:3000/demo/run?strategy=exponentialJitter&maxAttempts=  
3&failCount=2&baseDelayMs=500&multiplier=2&maxJitterMs=300"  
{  
  "success": true,  
  "totalAttempts": 3,  
  "delays": [768, 1235],  
  "successStatus": 200,  
  "finalResponse": {  
    "status": 200,  
    "body": {  
      "ok": true,  
      "attempt": 3,  
      "message": "Flaky service recovered"  
    }  
  }  
}
```

*Рис.5 – Робота Debounce при параметрах delayMs = 1000, leading = false
trailing = false*

Висновок

У ході виконання лабораторної роботи було досліджено принцип функціонування патерну Retry (Повтор) та розроблено його повнофункціональну реалізацію.

Було створено універсальний механізм автоматичного повторного виконання асинхронних операцій Retrier із підтримкою конфігурації максимальної кількості спроб maxAttempts та трьох стратегій розрахунку часових інтервалів: constant, exponential та exponentialJitter. Реалізація забезпечує точне дотримання лімітів очікування, динамічний розрахунок затримок за формулою експоненційного зростання, додавання псевдовипадкового шуму для запобігання проблемі Thundering Herd, а також фільтрацію винятків за допомогою параметра retryOn.

Реалізовано локальний HTTP-сервер на базі стандартного модуля node:http, який симулює нестабільні мережеві ендпоінти (/mock/flaky, /mock/fatal) і надає демонстраційний маршрут для ручної перевірки через утиліту curl.

У результаті проведених експериментів було успішно підтверджено коректність обробки всіх базових сценаріїв: негайне виконання з першої спроби, відновлення після транзитних серверних помилок 5xx із поступовим збільшенням пауз, коректне завершення роботи після вичерпання ліміту спроб, а також миттєве переривання без повторів у разі виникнення незворотних клієнтських помилок 4xx. Практичні результати довели високу ефективність патерну Retry для підвищення стійкості розподілених та хмарних систем до короточасних мережевих збоїв.

Код програми

Код до даної лабораторної роботи розміщений у репозиторії https://github.com/MrCatman137/Cloud_Technologies у директорії Lab_3 та має повну інструкцію запуску даної програми.